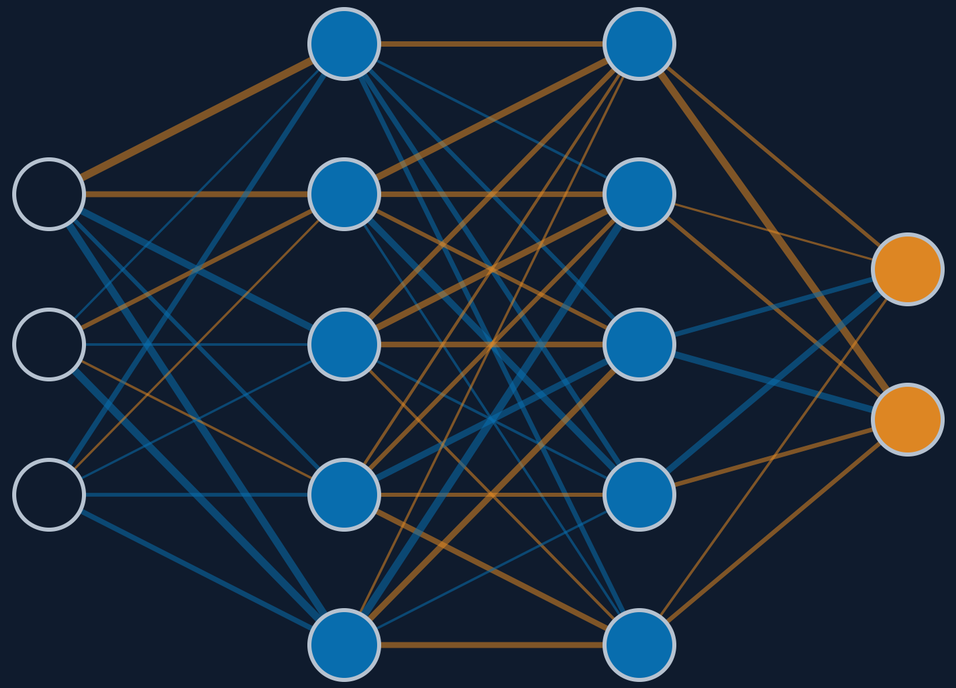
PORTFOLIO ARTIFACT 02 · NEURAL NETWORKS

Inside a Neural Network

How layers, neurons, weights, activations, loss and optimizers work together, tested hands-on in TensorFlow Playground



Student: Prathamesh Phutane

Course: 3FA2026 Model Development

ABOUT ME

About me and this artifact

About me

I am Prathamesh Phutane, a software engineer at Google with about three years of experience. I am now earning an M.S. in Artificial Intelligence at Indiana Wesleyan University. My goal is to build AI systems that people can trust and use every day.

Where this fits

Artifact 1, my Machine Learning Algorithm Framework, showed which algorithm to choose for a problem. **Artifact 2**, this deck, goes inside one of those algorithms: how a neural network is built and how it learns.

Skills shown in this artifact

- Neural network fundamentals
- TensorFlow Playground
- Experiment design
- Python and NumPy
- Data visualization
- Technical communication

Full portfolio: prathameshphutane.github.io

PROJECT DESCRIPTION

Context, challenge and solution

Context

Neural networks power image recognition, translation, chatbots and recommendations, but their inner workings are hard to picture.

Challenge

Explain six core components clearly, and show how changing each one affects what a model can learn and how fast.

Solution

A visual guide to every component, backed by 20 network setups in TensorFlow Playground, each trained 5 times (100 runs).

6 components explained

4 groups in one system

20 network setups tested

100 training runs

THE BIG PICTURE

How data flows through a network

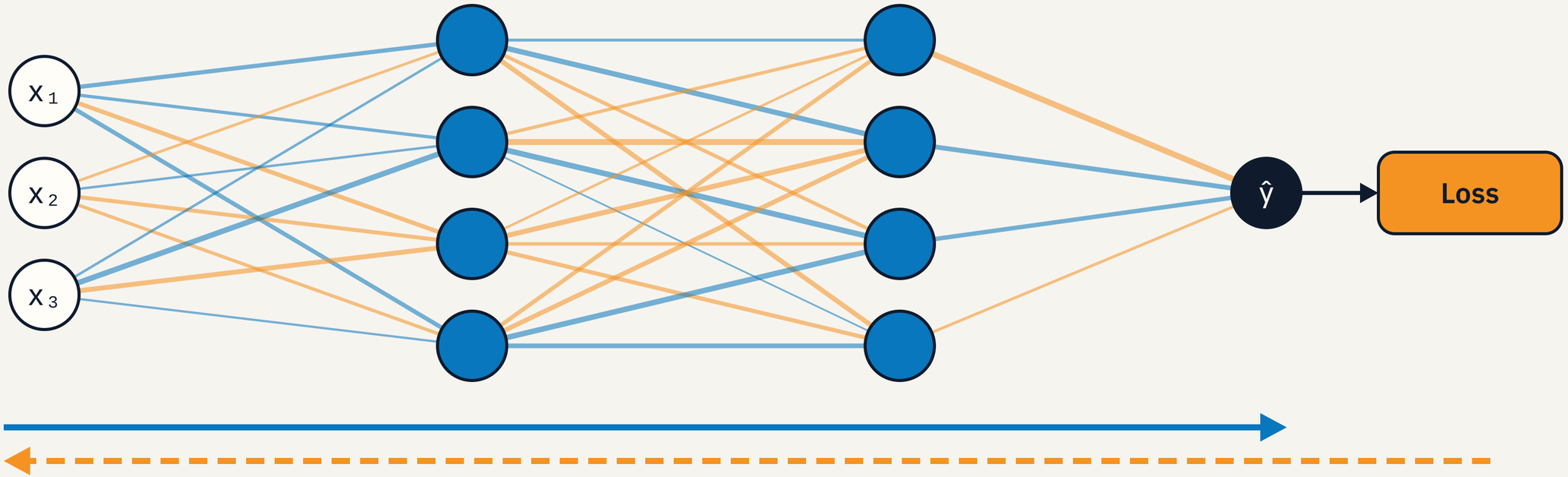
Input layer
features

Hidden layer 1
simple patterns

Hidden layer 2
combinations

Output layer
prediction ŷ

Loss
vs. true label

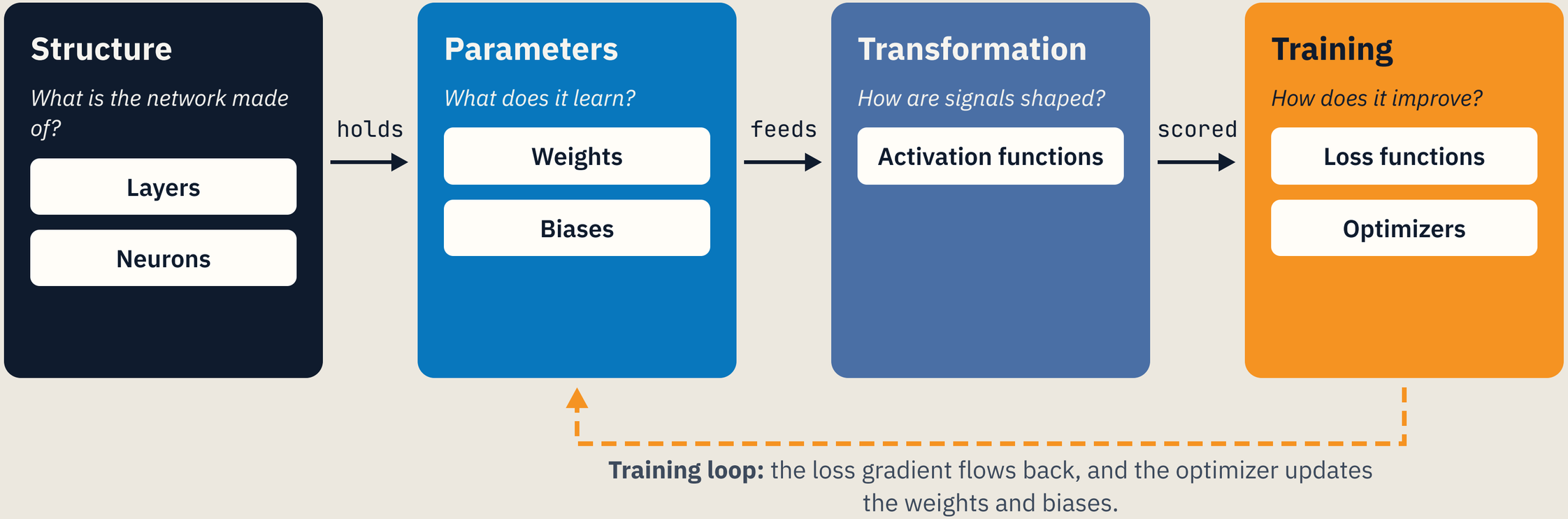


Forward pass → each layer computes weighted sums and activations until the output layer makes a prediction.

← **Backpropagation** (dashed orange): the loss gradient flows back so the optimizer can update every weight.

CLASSIFICATION

Four groups, one system



Training loop: the loss gradient flows back, and the optimizer updates the weights and biases.

COMPONENT 1 · LAYERS · STRUCTURE

Layers organize the network

A **layer** is a group of neurons that work at the same stage. Each layer takes the previous layer's output, transforms it, and passes it on. More layers let the network build complex ideas out of simple ones.

In my tests, spiral accuracy rose from **62%** with a [4, 2] network to **93%** with [8, 8, 8] (median of 5 runs).

Input layer

Holds the raw features, such as X_1 and X_2 in Playground. It does no calculation.

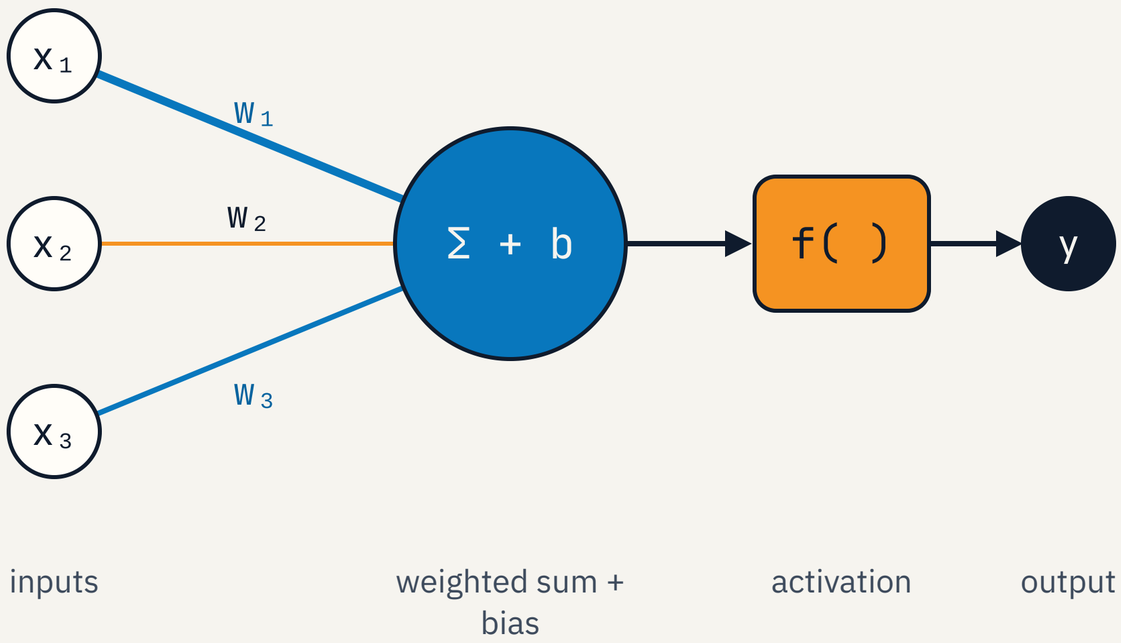
Hidden layers

Learn intermediate patterns: lines first, then curves and shapes built from them.

Output layer

Turns everything into the final answer, such as a class or a number.

A neuron is a tiny calculator



$$y = f(w_1x_1 + w_2x_2 + w_3x_3 + b)$$

1. Multiply each input by its weight.
2. Add the results and the bias.
3. Pass the sum through an activation function.
4. Send the output to the next layer.

In Playground, each neuron’s small square shows the part of the input space it responds to.

Weights decide what matters

A **weight** is the strength of a connection between two neurons. A large positive weight means “this input pushes the output up”; a negative weight pushes it down; a weight near zero means the input is ignored.

A **bias** shifts the neuron’s threshold, so it can activate even when its inputs are zero.

Weights start random (Playground uses values between −0.5 and 0.5) and are **learned** during training. Together they are the model’s memory.

25

learnable parameters in Playground’s default [4, 2] network with two inputs

177

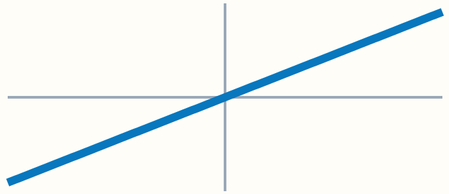
parameters in the [8, 8, 8] network that solved the spiral

In Playground, line thickness shows a weight’s size: **blue** = positive, **orange** = negative.

COMPONENT 4 · ACTIVATION FUNCTIONS · TRANSFORMATION

Activations add the curves

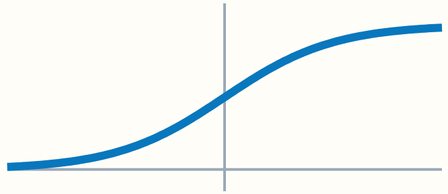
An activation function decides how strongly a neuron “fires.” Without a non-linear one, any stack of layers collapses into a single straight-line model.



Linear

`f(z) = z`

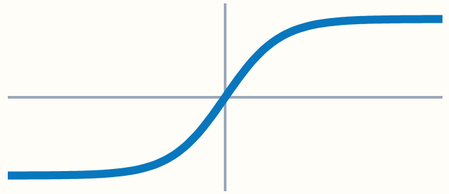
any value · no curve



Sigmoid

$1 / (1 + e^{-z})$

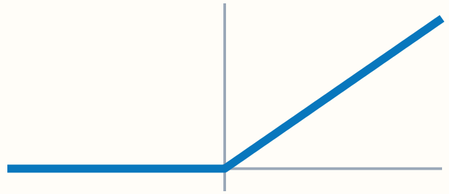
range 0 to 1



Tanh

`tanh(z)`

range –1 to 1



ReLU

`max(0, z)`

0 or positive

Spiral, [8, 8, 8] network: ReLU **96%**, tanh **93%**, linear and sigmoid about **60%**.

COMPONENT 5 · LOSS FUNCTIONS · TRAINING

Loss measures how wrong the model is

The loss function turns every prediction error into one number. Training is simply the search for weights that make this number as small as possible.

Mean squared error

$$(1/n) \Sigma (\hat{y} - y)^2$$

Used for regression. Big errors count much more than small ones. Playground uses this squared error for all of its tasks.

Cross-entropy

$$-[y \log \hat{y} + (1-y) \log(1-\hat{y})]$$

The usual choice for classification. It punishes confident wrong answers very hard.

Training vs. test loss

$$\text{gap} = \text{overfitting}$$

Spiral, [8, 8] network: training loss 0.015 but test loss 0.110. It memorized some points instead of learning the pattern.

COMPONENT 6 · OPTIMIZERS · TRAINING

Optimizers turn error into learning

1 **Forward pass**
make a prediction

→

2 **Compute loss**
measure the error

→

3 **Backpropagate**
find each weight's gradient

→

4 **Update**
nudge weights downhill

$$w \leftarrow w - \eta \cdot \partial L / \partial w$$

Move each weight a small step against its gradient. The **learning rate η** sets the step size.

Gradient descent

Uses the whole dataset for each step. It is stable but slow on big data.

Mini-batch SGD

Uses a small random batch per step. Playground uses batches of 10.

Adam

Adapts the step size for every weight. It is a common default today.

COMPARISON CHART

The six components side by side

Component	Category	Learned?	Role in a training step	Playground control
Layers	Structure	Chosen	Pass signals from input to output	+/– hidden layers
Neurons	Structure	Chosen	Weighted sum, then activation	+/– neurons
Weights & biases	Parameters	Learned	Carry the learned knowledge	Line color and thickness
Activation	Transformation	Chosen	Bend each neuron’s output	Activation menu
Loss	Training	Chosen	Score each prediction	Training and test loss
Optimizer	Training	Chosen	Update weights from gradients	Learning rate, batch size

Only the weights and biases are **learned**. Every other component is a design choice (a hyperparameter), which is why I could test each one in Playground.

CLASSIFICATION PROCESS

How I sorted the components

1. **List and define.** I wrote down each component and the one job it does in a training step.
2. **Group by question.** Parts that answer the same question went together: what it is made of, what it learns, how it shapes signals and how it improves.
3. **Test a second way.** I checked each part again: is it learned from data or chosen by the designer?
4. **Sort the experiments too.** Each Playground test was filed under the group it changes: data and inputs, structure, transformation or training.

> Sorting made me explain what each part *does*, not just name it. When a part seemed to fit two groups, that showed me where the groups connect.

Decisions on edge cases

Why not forward vs. backward pass?

Weights and activations are used in both passes, so that split put one part in two groups.

Biases go with weights

Both are learned numbers that the optimizer updates, so they share a group.

Neurons stay under structure

A neuron computes a sum, but its curve comes from the activation, so I kept the two apart.

HANDS-ON · TENSORFLOW PLAYGROUND

How I ran the experiments

What I changed	Values tested
Dataset	Gaussian, circle, XOR, spiral
Noise	0, 25, 50
Input features	X_1, X_2 · X_1^2, X_2^2 · all seven
Hidden layers	none, [2], [4, 2], [8, 8], [8, 8, 8]
Activation	linear, sigmoid, tanh, ReLU
Learning rate	0.003, 0.03, 0.3, 3

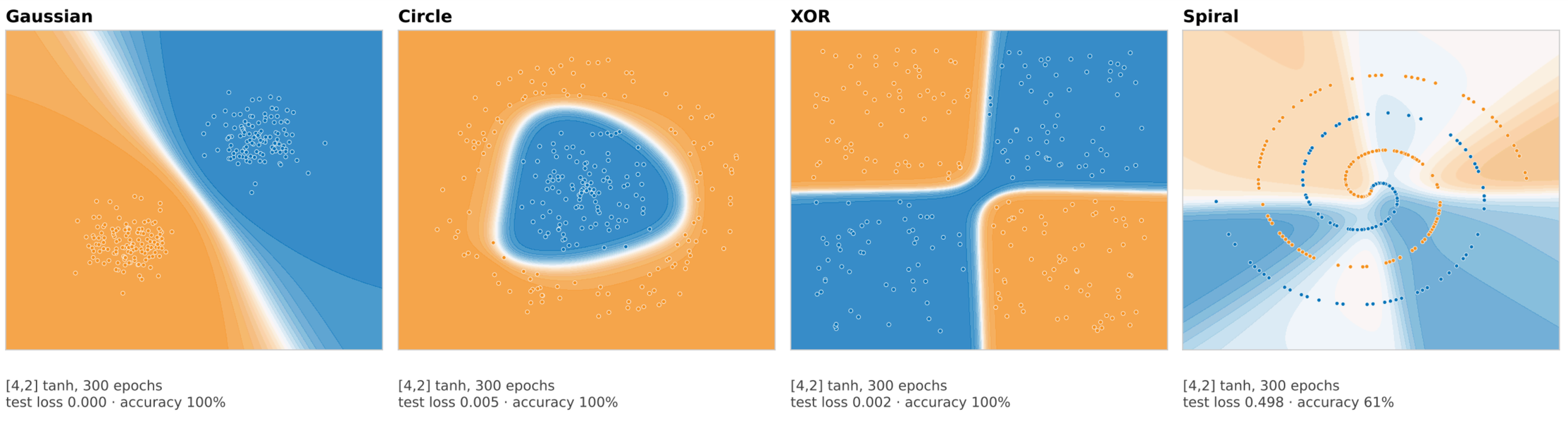
Kept the same: 500 points, 50% used for training, batch size 10, learning rate 0.03, tanh and a [4, 2] network, unless that setting was being tested.

Fair results: every setup was trained 5 times with different random starting weights. I report the median.

Exact numbers: I ran Playground's own training code in Python to log loss and accuracy for every run.

RESULT 1 · DATA TYPE

Harder patterns need more capacity



[4,2] tanh, 300 epochs
test loss 0.000 · accuracy 100%

[4,2] tanh, 300 epochs
test loss 0.005 · accuracy 100%

[4,2] tanh, 300 epochs
test loss 0.002 · accuracy 100%

[4,2] tanh, 300 epochs
test loss 0.498 · accuracy 61%

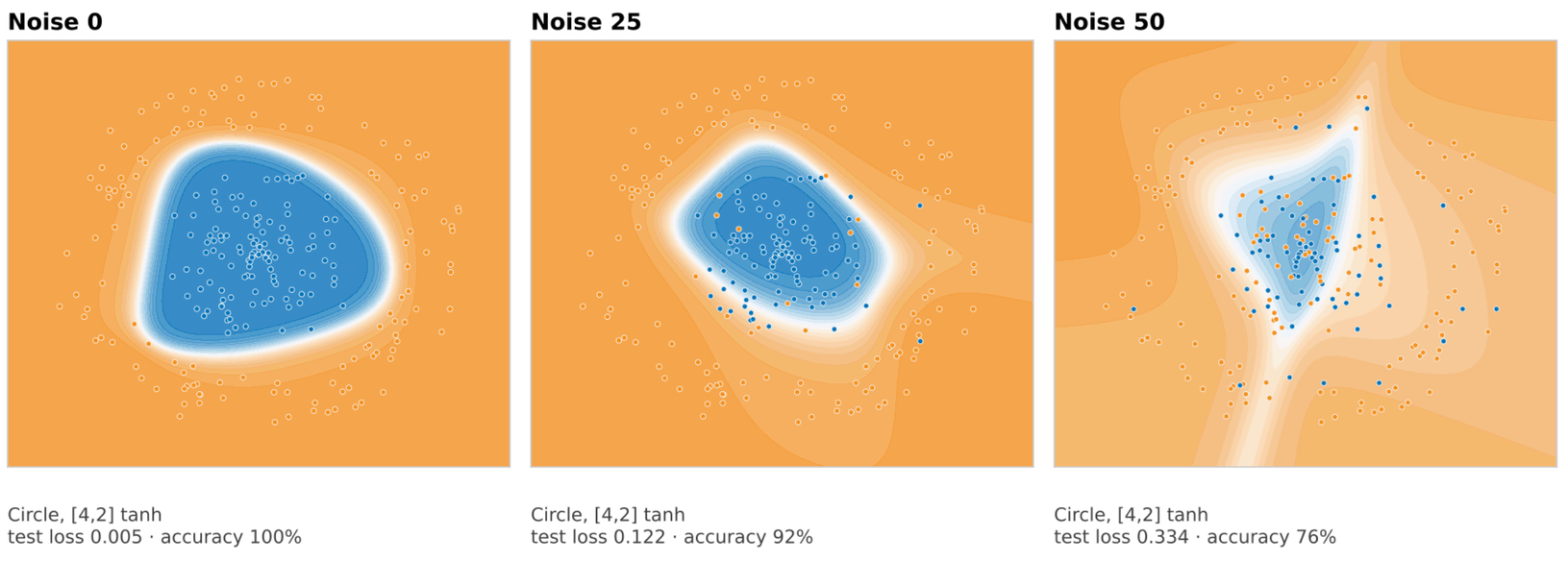
3 epochs to solve the Gaussian clusters, the easiest dataset

41–57 epochs for XOR and circle: curves take longer to learn

61% spiral accuracy: the default network is too small

RESULT 2 · NOISE

Noise sets a ceiling on accuracy



Circle, [4,2] tanh
test loss 0.005 · accuracy 100%

Circle, [4,2] tanh
test loss 0.122 · accuracy 92%

Circle, [4,2] tanh
test loss 0.334 · accuracy 76%

100%
noise 0

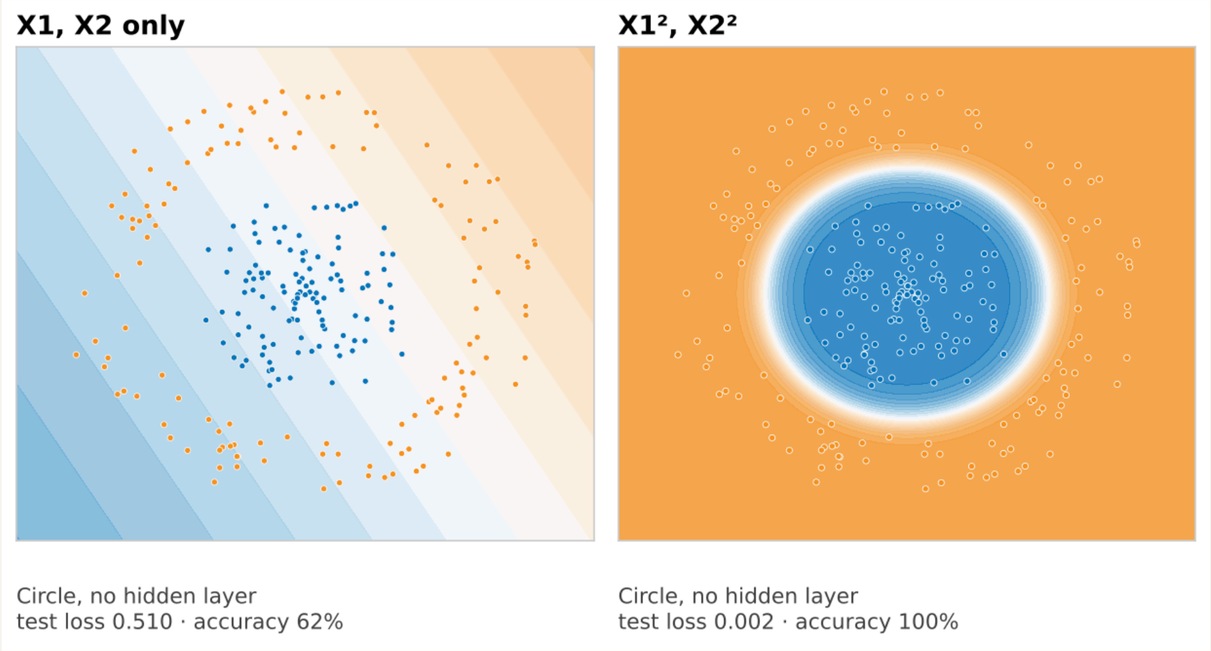
92%
noise 25

74%
noise 50

Noise mixes up the labels near the boundary. No network can predict a mislabeled point, so even a perfect model loses accuracy.

RESULT 3 · INPUT FEATURES

The right features can replace layers



No hidden layer, X_1 and X_2

A single neuron can only draw a straight line: 55% on the circle.

No hidden layer, X_1^2 and X_2^2

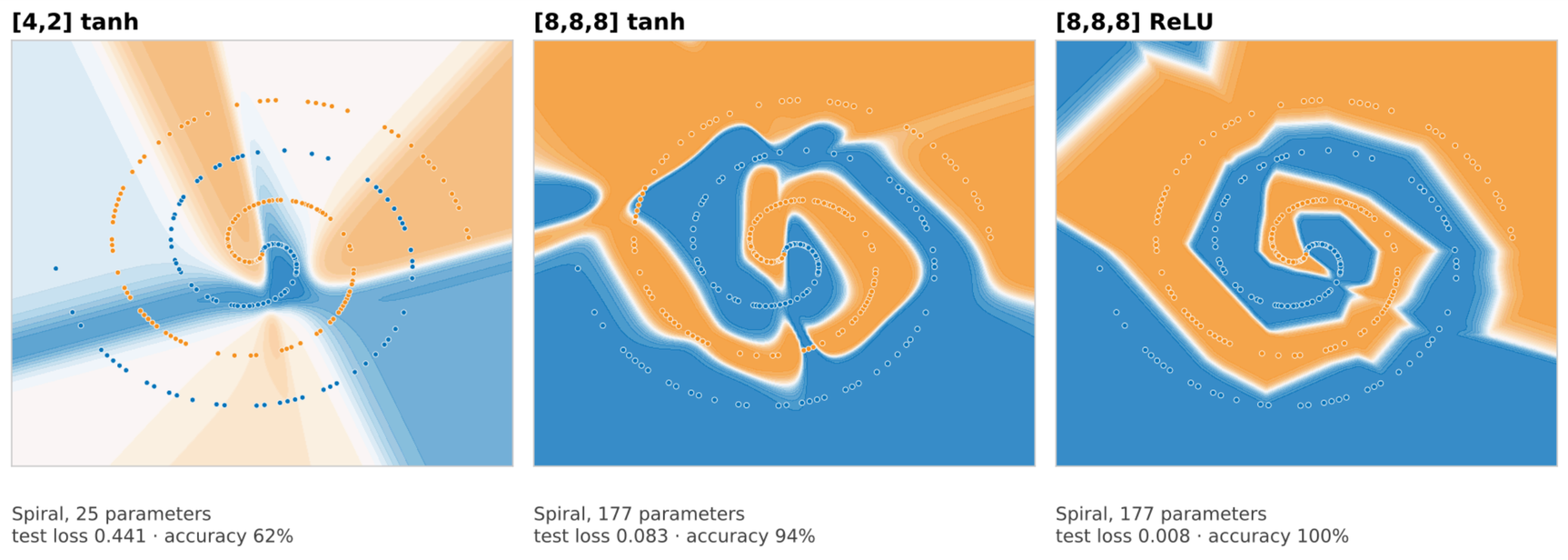
Squared inputs turn the circle into a straight cut: 100% with only 3 parameters.

Spiral with all 7 features

80% instead of 61% with the same [4, 2] network.

RESULT 4 · NEURONS AND LAYERS

More neurons and layers can learn harder shapes



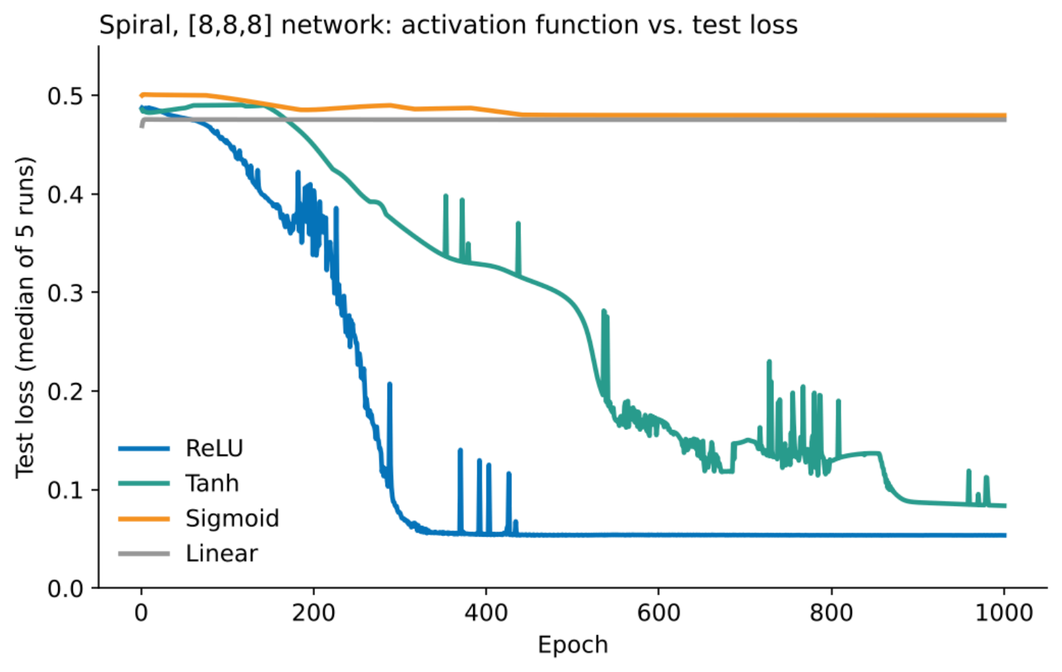
62% [4, 2] tanh, 25 parameters

93% [8, 8, 8] tanh, 177 parameters

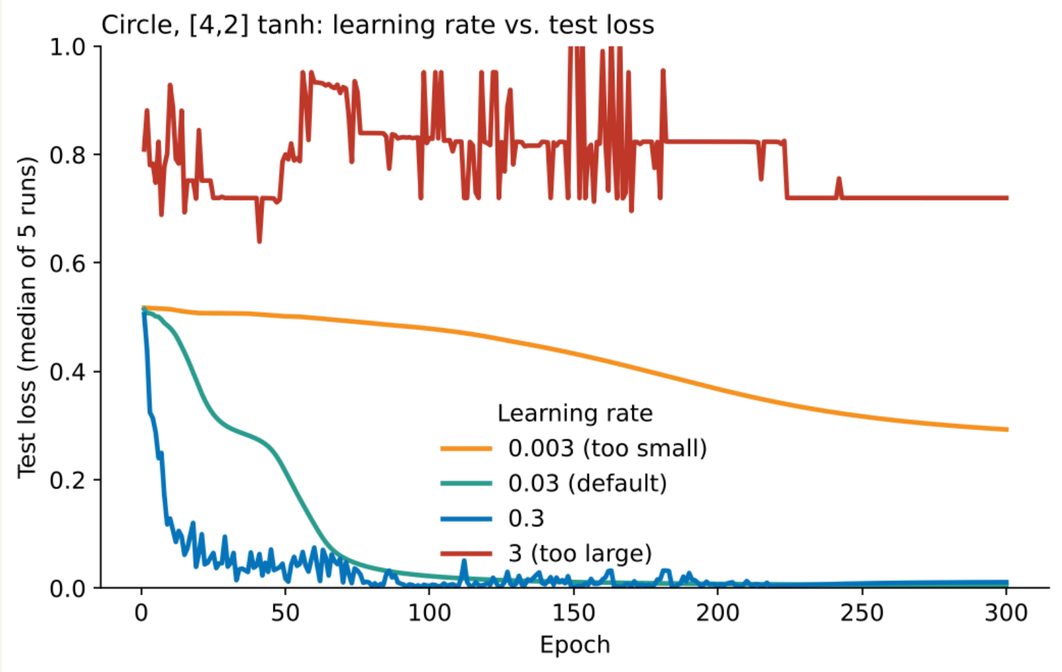
96% [8, 8, 8] ReLU, and about 2.5× faster than tanh

RESULT 5 · ACTIVATION AND LEARNING RATE

Small settings, big differences in training



ReLU learned fastest. Linear could not bend the boundary, and sigmoid stalled.



A rate of **0.3** converged in 12 epochs. A rate of 0.003 crawled, and 3 bounced around and never converged.

ALL RESULTS

What 100 training runs showed

Experiment	Group	Settings compared	Median accuracy	Lesson
Data type	Data and inputs	Gaussian / circle / XOR / spiral	100 / 100 / 99 / 61%	Hard shapes need capacity
Noise	Data and inputs	0 / 25 / 50	100 / 92 / 74%	Noisy labels cap accuracy
Features	Data and inputs	Raw vs. squared inputs	55 → 100%	Good features beat layers
Layers	Structure	[4, 2] vs. [8, 8, 8]	62 → 93%	Capacity solves the spiral
Activation	Transformation	linear / sigmoid / tanh / ReLU	60 / 59 / 93 / 96%	Non-linearity is essential
Learning rate	Training	0.003 / 0.03 / 0.3 / 3	84 / 100 / 99 / 64%	Pick a middle step size

Test accuracy, median of 5 runs per setting. Layers and activation: spiral, 1,000 epochs. Others: 300 epochs.

SUMMARY

Why visualizing neural networks matters

Why visualize

- A diagram shows how data and errors move, which equations alone hide.
- Live boundaries reveal what a model has learned, and where it fails.
- Visuals make AI explainable to people who do not write code.

Key insights

- Every component has a job: layers and neurons give capacity, weights store knowledge, activations add curves, loss measures error and optimizers drive the learning.
- The right input features can matter more than adding layers.
- The learning rate and the activation function can make training several times faster or slower.
- Noise sets a limit that no model can beat, and random starting weights make results vary.